

Who Am I?

My name is **Onur TURAN**, and I am a student in the Mathematical Engineering program at Yıldız Technical University. I have a strong security background, with experience in cybersecurity and ethical hacking. I am passionate about learning new technologies and applying them to solve real-world problems. In this course, I am eager to deepen my understanding of data management and file structures, and I look forward to working on the project that will allow me to design a complete database system for a Smart Healthcare Platform for this lecture homework.

- Name: Onur TURAN
- University: Yıldız Technical University
- Department: Mathematical Engineering
- Number: 19058006

Contents

1	1. Requirements Analysis	4
2	2. ER/EER Model Design	8
3	Diagram Checklist	12
4	3. Relational Schema Mapping	14
5	4. Functional Dependencies	16
6	5. Normalization Analysis	17
7	6. SQL Implementation	19
8	7. Advanced SQL Queries	26
9	8. Relational Algebra and Calculus	29
10	9. Indexing and File Structures	30

Project Objective

Design and implement a complete database system for a Smart Healthcare Platform that supports:

- Multi-user access (patients, doctors, admins)
- Complex data relationships (EER level)
- Query optimization and indexing
- Real-world constraints and normalization

This project directly satisfies course outcomes like:

- ER/EER modeling
- Relational schema design
- Functional dependencies & normalization
- SQL querying
- File structures & indexing

Project Scenario

You are developing a database system for a large hospital network that includes:

- Multiple hospitals and departments
- Doctors with different specialties
- Patients with medical histories
- Appointments, prescriptions, and lab results
- Insurance and billing system

The system must support efficient querying, integrity, and scalability.

1 1. Requirements Analysis

1.1 System Overview

The Smart Healthcare Management System is a comprehensive database solution designed to support large-scale hospital networks with multiple facilities, departments, healthcare providers, and patients. The system manages complex healthcare workflows including patient records, medical appointments, prescriptions, laboratory results, insurance claims, and billing operations.

1.2 Entity Requirements

The system must model the following key entities:

Organizational Entities

- **Hospital:** Multiple hospital branches with location, contact information, and operational details
- **Department:** Specialized departments within hospitals (Cardiology, Neurology, Orthopedics, etc.)
- **User:** Superclass representing all system users with authentication and profile information

Healthcare Professional Entities

- **Doctor:** Specialized subclass of User with license number, specialty, department assignment, and consultation fees
- **Admin:** Administrative staff subclass with role and office contact details

Patient and Medical Entities

- **Patient:** Patient subclass with blood type, emergency contact, and medical history tracking
- **Appointment:** Clinical encounters between patients and doctors with diagnosis and status tracking
- **Prescription:** Medication orders issued during appointments (weak entity)
- **LabResult:** Laboratory test results and clinical findings
- **Diagnosis:** Medical diagnoses using ICD codes

Insurance and Billing Entities

- **Insurance:** Patient insurance policy information and coverage details
- **Billing/Invoice:** Financial transactions, payment status, and claim management

Security and Device Management

- **MedicalDevice:** Medical equipment inventory with firmware and risk assessment
- **SecurityEvent:** Audit logs for system access, device interactions, and security monitoring

1.3 Relationship Requirements

The system must support the following relationships:

Relationship	Type	Cardinality
Hospital → Department	Organizational	1:N
Department → Doctor	Assignment	1:N
Hospital → MedicalDevice	Equipment	1:N
Patient → Appointment	Medical History	1:N
Doctor → Appointment	Service Provider	1:N
Appointment → Prescription	Medical Order	1:N
Appointment → LabResult	Clinical Finding	1:N
Appointment → Diagnosis	ICD Coding	1:N
Patient → Insurance	Coverage	1:N
Appointment → Invoice	Billing	1:N
User → SecurityEvent	Audit Trail	1:N

1.4 Data Constraints

Uniqueness Constraints

- User email addresses must be globally unique
- Doctor license numbers must be unique
- Insurance policy numbers must be unique
- Department names must be unique within a hospital

Domain Constraints

- User role must be one of: PATIENT, DOCTOR, ADMIN, SECURITY_ANALYST
- Appointment status must be: SCHEDULED, COMPLETED, CANCELLED
- Invoice status must be: UNPAID, PARTIAL, PAID

- Gender must be: M, F, or O
- Consultation fees must be non-negative
- Patient blood type must be: A+, A-, B+, B-, AB+, AB-, O+, O-
- Risk levels for devices: LOW, MEDIUM, HIGH, CRITICAL
- Event severity: INFO, WARNING, ERROR, CRITICAL

Referential Integrity

- All foreign keys must maintain referential integrity with cascade deletion where appropriate
- Patient records cannot be deleted if appointments exist (protect historical data)
- Doctor records must reference valid users
- Department records must reference valid hospitals

1.5 Specialization/Generalization (EER)

The User entity employs a **total, disjoint specialization** hierarchy:

- **Superclass:** User (supertype)
- **Subclasses:** Patient, Doctor, Admin
- **Specialization Type:** Disjoint (a user cannot be both Patient and Doctor simultaneously)
- **Participation:** Total (every user must have a role)
- **Discriminator:** user_type attribute

1.6 Weak Entity

The **Prescription** entity is a weak entity:

- **Owner Entity:** Appointment
- **Identifying Relationship:** Each prescription must reference an appointment
- **Partial Key:** prescription_no (unique within an appointment only)
- **Full Key:** (appointment_id, prescription_no)

1.7 Multivalued Attributes

- **User.phone:** Users may have multiple phone numbers (mapped to separate table)
- **Patient.phone:** Patients may have multiple contact numbers

1.8 Composite Attributes

- **User.Name:** FirstName, LastName
- **User.Address:** Street, City, PostalCode

1.9 Non-Functional Requirements

Performance

- Appointment searches by patient or date must complete in $< 100\text{ms}$ for datasets with $1\text{M}+$ records
- Security event queries must support real-time SIEM integration
- Indexing strategy required for frequently accessed columns

Security

- Patient medical records must be audit-logged
- All data modifications must be tracked with timestamp and user context
- System must support role-based access control (RBAC)
- Sensitive data (SSN, insurance details) must be encrypted

Integrity

- Appointment cannot be created for doctors not in the appointment's department
- Doctor must be assigned to a department before creating appointments
- Invoice must reference a valid appointment
- Prescription must reference a valid medication

Scalability

- System must support multiple hospital networks (multi-tenancy ready)
- Database must scale horizontally to support $> 100,000$ concurrent users
- Partition strategy for historical appointment data recommended

2 2. ER/EER Model Design

2.1 Requirements Analysis Summary

The Smart Healthcare Platform must support:

- Multiple hospitals and departments
- Doctors with specialties and licenses
- Patients with medical histories and insurance
- Appointments and medical records
- Prescriptions (weak entity dependent on appointments)
- Lab results and billing
- Security and device management

2.2 Entity Definitions

1. Hospital

Attribute	Type	Notes
HospitalID	PK	
HospitalName	VARCHAR	
Address	VARCHAR	
City	VARCHAR	
PhoneNumber	VARCHAR	

2. User (Superclass for EER)

Attribute	Type	Notes
UserID	PK	
FirstName	VARCHAR	composite: Name
LastName	VARCHAR	composite: Name
Email	VARCHAR	UNIQUE
Phone	VARCHAR	multivalued
City	VARCHAR	composite: Address
Street	VARCHAR	composite: Address
PostalCode	VARCHAR	composite: Address
Role	VARCHAR	Patient, Doctor, Admin, SecurityAnalyst

2.3 Detailed Entity Specifications

3. Department

Relationship: one hospital $\rightarrow$ many departments.

Attribute	Type	Notes
DepartmentID	PK	
HospitalID	FK	references Hospital
DepartmentName	VARCHAR	
FloorNumber	INTEGER	

4. Patient

Attribute	Type	Notes
PatientID	PK	
UserID	FK	references User
BloodType	VARCHAR	
BirthDate	DATE	
EmergencyContact	VARCHAR	
InsuranceID	FK	references Insurance

5. Doctor

Attribute	Type	Notes
DoctorID	PK	
UserID	FK	references User
DepartmentID	FK	
Specialty	VARCHAR	
LicenseNumber	VARCHAR	UNIQUE

6. Appointment

This works like a junction entity.

It resolves the Doctor $\leftrightarrow$ Patient relationship.

Attribute	Type	Notes
AppointmentID	PK	
PatientID	FK	
DoctorID	FK	
AppointmentDate	TIMESTAMP	
Status	VARCHAR	
DiagnosisSummary	TEXT	

7. Prescription (Weak Entity)

Weak entity.

It is meaningless without an Appointment.

Attribute	Type	Notes
PrescriptionNo	Partial Key	
AppointmentID	FK	identifying relationship
MedicineName	VARCHAR	
Dosage	VARCHAR	
DurationDays	INTEGER	

Composite PK:

(PrescriptionNo, AppointmentID)

8. LabResult

Attribute	Type	Notes
ResultID	PK	
AppointmentID	FK	
TestName	VARCHAR	
ResultValue	VARCHAR	
ResultDate	DATE	

9. Insurance

Attribute	Type	Notes
InsuranceID	PK	
ProviderName	VARCHAR	
PolicyNumber	VARCHAR	
CoverageType	VARCHAR	

10. Billing

Attribute	Type	Notes
BillID	PK	
PatientID	FK	
Amount	DECIMAL	
PaymentStatus	VARCHAR	
BillingDate	DATE	

11. MedicalDevice

Cybersecurity starts here.

Attribute	Type	Notes
DeviceID	PK	
HospitalID	FK	
DeviceType	VARCHAR	
FirmwareVersion	VARCHAR	
RiskLevel	VARCHAR	

12. SecurityEvent

SIEM logic.

Attribute	Type	Notes
EventID	PK	
UserID	FK	
DeviceID	FK	nullable
EventType	VARCHAR	
Severity	VARCHAR	
SourceIP	VARCHAR	
EventTimestamp	TIMESTAMP	
Composite PK:		

(*DeviceID*, *VulnerabilityID*)

ER Relationship Summary

Relationship	Cardinality
Hospital → Department	1:N
Department → Doctor	1:N
Patient → Appointment	1:N
Doctor → Appointment	1:N
Appointment → Prescription	1:N
Appointment → LabResult	1:N
Patient → Billing	1:N
Hospital → MedicalDevice	1:N
MedicalDevice ↔ Vulnerability	N:M
User → SecurityEvent	1:N

EER Model: Enhanced Features

Overview

Now the EER part.

Specialization / Generalization

Superclass

User

Subclasses Patient Doctor Admin SecurityAnalyst

This structure shows:

inheritance specialization ISA hierarchy

EER Features

EER Özellikleri Type:

Disjoint Specialization

A user cannot be both:
the Patient the Doctor
at the same time.

Participation:
Total Participation
Every User must have a role.

Composite Attribute Example

User:
Name:
FirstName LastName
Address:
City Street PostalCode

Multivalued Attribute

Patient:
PhoneNumbers
During relational mapping, this is separated into its own table:
PatientPhone
PatientID PhoneNumber

Weak Entity

Prescription
Owner entity:
Appointment
Partial key:
PrescriptionNo

Category / Union Type (Optional Bonus)

StaffAccess
Union:
Doctor Admin SecurityAnalyst
It is optional; do not overcomplicate it.

3 Diagram Checklist

ER diagram should show:

- PK
- FK
- cardinality
- weak entity

- total participation

EER diagram should additionally show:

- ISA hierarchy
- specialization
- inheritance
- disjoint/overlap
- total/partial participation

This structure is enough for the ER/EER part of the assignment.

4 3. Relational Schema Mapping

Convert the ER/EER design into normalized relational tables:

3.1 Relational Schema

User (UserID, FirstName, LastName, Email, Phone, City, Street, PostalCode, Role)

- PK: UserID
- Composite attribute: Name (FirstName, LastName), Address (City, Street, PostalCode)
- Multivalued Phone handled in UserPhone table

Hospital (HospitalID, HospitalName, Address, City, PhoneNumber)

- PK: HospitalID

Department (DepartmentID, HospitalID, DepartmentName, FloorNumber)

- PK: DepartmentID
- FK: HospitalID → Hospital

Doctor (DoctorID, UserID, DepartmentID, Specialty, LicenseNumber)

- PK: DoctorID
- FK: UserID → User
- FK: DepartmentID → Department
- UNIQUE: LicenseNumber

Patient (PatientID, UserID, BloodType, BirthDate, EmergencyContact, InsuranceID)

- PK: PatientID
- FK: UserID → User
- FK: InsuranceID → Insurance

Appointment (AppointmentID, PatientID, DoctorID, AppointmentDate, Status, DiagnosisSummary)

- PK: AppointmentID
- FK: PatientID → Patient
- FK: DoctorID → Doctor

Prescription (PrescriptionNo, AppointmentID, MedicineName, Dosage, DurationDays)

- PK: (PrescriptionNo, AppointmentID) — Composite Key
- FK: AppointmentID $\rightarrow$ Appointment
- Weak entity dependent on Appointment

LabResult (ResultID, AppointmentID, TestName, ResultValue, ResultDate)

- PK: ResultID
- FK: AppointmentID $\rightarrow$ Appointment

Insurance (InsuranceID, ProviderName, PolicyNumber, CoverageType)

- PK: InsuranceID

Billing (BillID, PatientID, Amount, PaymentStatus, BillingDate)

- PK: BillID
- FK: PatientID $\rightarrow$ Patient

MedicalDevice (DeviceID, HospitalID, DeviceType, FirmwareVersion, RiskLevel)

- PK: DeviceID
- FK: HospitalID $\rightarrow$ Hospital

SecurityEvent (EventID, UserID, DeviceID, EventType, Severity, SourceIP, EventTimestamp)

- PK: EventID
- FK: UserID $\rightarrow$ User
- FK: DeviceID $\rightarrow$ MedicalDevice (nullable)

UserPhone (UserID, PhoneNumber)

- PK: (UserID, PhoneNumber)
- FK: UserID $\rightarrow$ User

5 4. Functional Dependencies

4.1 Hospital Table

- $\text{HospitalID} \rightarrow \text{HospitalName}, \text{Address}, \text{City}, \text{PhoneNumber}$
- All attributes depend solely on HospitalID

4.2 Doctor Table

- $\text{DoctorID} \rightarrow \text{UserID}, \text{DepartmentID}, \text{Specialty}, \text{LicenseNumber}$
- $\text{UserID} \rightarrow \text{FirstName}, \text{LastName}, \text{Email}$ (via User table)
- $\text{DepartmentID} \rightarrow \text{DepartmentName}, \text{FloorNumber}$ (via Department table)
- $\text{LicenseNumber} \rightarrow \text{DoctorID}$ (candidate key)

4.3 Patient Table

- $\text{PatientID} \rightarrow \text{UserID}, \text{BloodType}, \text{BirthDate}, \text{EmergencyContact}, \text{InsuranceID}$
- $\text{UserID} \rightarrow \text{FirstName}, \text{LastName}, \text{Email}$ (via User table)
- $\text{InsuranceID} \rightarrow \text{ProviderName}, \text{PolicyNumber}, \text{CoverageType}$ (via Insurance table)

4.4 Appointment Table

- $\text{AppointmentID} \rightarrow \text{PatientID}, \text{DoctorID}, \text{AppointmentDate}, \text{Status}, \text{DiagnosisSummary}$
- $\text{PatientID} \rightarrow \text{BloodType}, \text{BirthDate}$ (via Patient table)
- $\text{DoctorID} \rightarrow \text{Specialty}, \text{LicenseNumber}$ (via Doctor table)

4.5 Prescription Table

- $(\text{PrescriptionNo}, \text{AppointmentID}) \rightarrow \text{MedicineName}, \text{Dosage}, \text{Duration-Days}$
- $\text{AppointmentID} \rightarrow \text{PatientID}, \text{DoctorID}$ (via Appointment table)

6 5. Normalization Analysis

5.1 First Normal Form (1NF)

Status: ✓ Already in 1NF

All tables have atomic values (no composite or multivalued attributes in main tables).

- User: Phone moved to UserPhone table
- Address attributes (City, Street, PostalCode) kept as separate columns (acceptable for 1NF)

5.2 Second Normal Form (2NF)

Status: ✓ Already in 2NF

All non-key attributes are fully dependent on the primary key.

- Doctor table: DoctorID is PK; Specialty and LicenseNumber depend fully on DoctorID
- Patient table: PatientID is PK; BloodType, BirthDate depend fully on PatientID
- Prescription table: Composite PK (PrescriptionNo, AppointmentID); MedicineName, Dosage depend on the full key

5.3 Third Normal Form (3NF)

Status: ✓ Already in 3NF

No transitive dependencies. Non-key attributes do not depend on other non-key attributes.

- Doctor: Specialty depends directly on DoctorID, not on another non-key attribute
- Patient: BloodType depends directly on PatientID
- Appointment: Status depends on AppointmentID, not transitively through PatientID

5.4 Boyce-Codd Normal Form (BCNF)

Status: ✓ BCNF-compliant for critical tables

Every determinant is a candidate key.

- Doctor: LicenseNumber is a candidate key; all FDs originate from candidate keys
- Hospital: HospitalID is the only determinant and is the PK
- Insurance: InsuranceID is the only determinant and is the PK

5.5 Normalization Summary Table

Table	1NF	2NF	3NF	BCNF
Hospital	✓	✓	✓	✓
Doctor	✓	✓	✓	✓
Patient	✓	✓	✓	✓
Appointment	✓	✓	✓	✓
Prescription	✓	✓	✓	✓
Department	✓	✓	✓	✓

All tables are normalized up to BCNF, ensuring data integrity and minimizing redundancy.

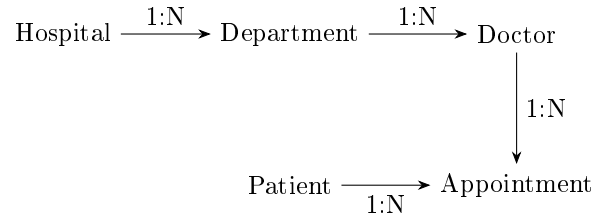


Figure 1: ER özeti (modüler)

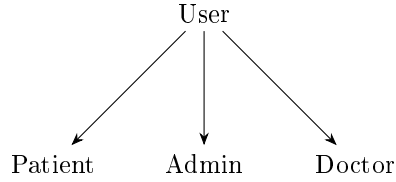


Figure 2: EER ISA (modüler)

7 6. SQL Implementation

6.1 CREATE TABLE Statements

```
CREATE TABLE Hospital (  
    HospitalID INT PRIMARY KEY AUTO_INCREMENT,  
    HospitalName VARCHAR(100) NOT NULL,  
    Address VARCHAR(150),  
    City VARCHAR(50),  
    PhoneNumber VARCHAR(15)  
);  
  
CREATE TABLE User (  
    UserID INT PRIMARY KEY AUTO_INCREMENT,  
    FirstName VARCHAR(50) NOT NULL,  
    LastName VARCHAR(50) NOT NULL,  
    Email VARCHAR(100) UNIQUE NOT NULL,  
    City VARCHAR(50),  
    Street VARCHAR(100),  
    PostalCode VARCHAR(10),  
    Role ENUM('Patient', 'Doctor', 'Admin', 'SecurityAnalyst')  
);  
  
CREATE TABLE UserPhone (  
    UserID INT NOT NULL,  
    PhoneNumber VARCHAR(15) NOT NULL,  
    PRIMARY KEY (UserID, PhoneNumber),  
    FOREIGN KEY (UserID) REFERENCES User(UserID) ON DELETE CASCADE  
);  
  
CREATE TABLE Insurance (  
    InsuranceID INT PRIMARY KEY AUTO_INCREMENT,  
    ProviderName VARCHAR(100) NOT NULL,  
    PolicyNumber VARCHAR(50) UNIQUE,  
    CoverageType VARCHAR(50)  
);  
  
CREATE TABLE Patient (  
    PatientID INT PRIMARY KEY AUTO_INCREMENT,  
    UserID INT NOT NULL UNIQUE,  
    BloodType VARCHAR(5),  
    BirthDate DATE,  
    EmergencyContact VARCHAR(100),  
    InsuranceID INT,  
    FOREIGN KEY (UserID) REFERENCES User(UserID) ON DELETE CASCADE,  
    FOREIGN KEY (InsuranceID) REFERENCES Insurance(InsuranceID)
```

```

);

CREATE TABLE Department (
    DepartmentID INT PRIMARY KEY AUTO_INCREMENT,
    HospitalID INT NOT NULL,
    DepartmentName VARCHAR(100) NOT NULL,
    FloorNumber INT,
    FOREIGN KEY (HospitalID) REFERENCES Hospital(HospitalID) ON DELETE CASCADE
);

CREATE TABLE Doctor (
    DoctorID INT PRIMARY KEY AUTO_INCREMENT,
    UserID INT NOT NULL UNIQUE,
    DepartmentID INT NOT NULL,
    Specialty VARCHAR(100),
    LicenseNumber VARCHAR(50) UNIQUE NOT NULL,
    FOREIGN KEY (UserID) REFERENCES User(UserID) ON DELETE CASCADE,
    FOREIGN KEY (DepartmentID) REFERENCES Department(DepartmentID)
);

CREATE TABLE Appointment (
    AppointmentID INT PRIMARY KEY AUTO_INCREMENT,
    PatientID INT NOT NULL,
    DoctorID INT NOT NULL,
    AppointmentDate TIMESTAMP,
    Status VARCHAR(50),
    DiagnosisSummary TEXT,
    FOREIGN KEY (PatientID) REFERENCES Patient(PatientID) ON DELETE CASCADE,
    FOREIGN KEY (DoctorID) REFERENCES Doctor(DoctorID),
    INDEX idx_patient (PatientID),
    INDEX idx_doctor (DoctorID),
    INDEX idx_date (AppointmentDate)
);

CREATE TABLE Prescription (
    PrescriptionNo INT NOT NULL,
    AppointmentID INT NOT NULL,
    MedicineName VARCHAR(100),
    Dosage VARCHAR(50),
    DurationDays INT,
    PRIMARY KEY (PrescriptionNo, AppointmentID),
    FOREIGN KEY (AppointmentID) REFERENCES Appointment(AppointmentID) ON DELETE CASCADE
);

CREATE TABLE LabResult (
    ResultID INT PRIMARY KEY AUTO_INCREMENT,

```

```

        AppointmentID INT NOT NULL,
        TestName VARCHAR(100),
        ResultValue VARCHAR(100),
        ResultDate DATE,
        FOREIGN KEY (AppointmentID) REFERENCES Appointment(AppointmentID) ON DELETE CASCADE
    );

CREATE TABLE Billing (
    BillID INT PRIMARY KEY AUTO_INCREMENT,
    PatientID INT NOT NULL,
    Amount DECIMAL(10, 2),
    PaymentStatus VARCHAR(50),
    BillingDate DATE,
    FOREIGN KEY (PatientID) REFERENCES Patient(PatientID) ON DELETE CASCADE
);

CREATE TABLE MedicalDevice (
    DeviceID INT PRIMARY KEY AUTO_INCREMENT,
    HospitalID INT NOT NULL,
    DeviceType VARCHAR(100),
    FirmwareVersion VARCHAR(50),
    RiskLevel VARCHAR(20),
    FOREIGN KEY (HospitalID) REFERENCES Hospital(HospitalID) ON DELETE CASCADE
);

CREATE TABLE SecurityEvent (
    EventID INT PRIMARY KEY AUTO_INCREMENT,
    UserID INT NOT NULL,
    DeviceID INT,
    EventType VARCHAR(100),
    Severity VARCHAR(50),
    SourceIP VARCHAR(15),
    EventTimestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (UserID) REFERENCES User(UserID) ON DELETE CASCADE,
    FOREIGN KEY (DeviceID) REFERENCES MedicalDevice(DeviceID) ON DELETE SET NULL,
    INDEX idx_user (UserID),
    INDEX idx_timestamp (EventTimestamp)
);

```

6.2 Sample INSERT Statements

```

-- Insert sample hospitals
INSERT INTO Hospital VALUES
(1, 'Central City Hospital', '123 Main St', 'Istanbul', '2125551234'),
(2, 'Metropolitan Medical Center', '456 Oak Ave', 'Ankara', '3125556789'),
(3, 'Aegean Health Campus', '789 Coastal Rd', 'Izmir', '2325553456');

```

```

-- Insert 15+ User records (mix of doctors, patients, admins)
INSERT INTO User VALUES
(1, 'Ahmet', 'Yilmaz', 'ahmet.yilmaz@mail.com', 'Istanbul', 'Main St', '34000', 'Doctor'),
(2, 'Fatima', 'Kaya', 'fatima.kaya@mail.com', 'Istanbul', 'Oak Ave', '34001', 'Patient'),
(3, 'Mehmet', 'Demir', 'mehmet.demir@mail.com', 'Ankara', 'Central Ave', '06100', 'Admin'),
(4, 'Leyla', 'Gunes', 'leyla.gunes@mail.com', 'Istanbul', 'Park St', '34002', 'Doctor'),
(5, 'Ali', 'Ozturk', 'ali.ozturk@mail.com', 'Ankara', 'Peace Rd', '06101', 'Patient'),
(6, 'Ayse', 'Kaya', 'ayse.kaya@mail.com', 'Izmir', 'Beach Ave', '35000', 'Patient'),
(7, 'Kemal', 'Sezer', 'kemal.sezer@mail.com', 'Istanbul', 'Hill St', '34003', 'Doctor'),
(8, 'Nur', 'Acar', 'nur.acar@mail.com', 'Ankara', 'Green Rd', '06102', 'Doctor'),
(9, 'Zeynep', 'Koc', 'zeynep.koc@mail.com', 'Izmir', 'Port Rd', '35001', 'Patient'),
(10, 'Can', 'Arslan', 'can.arslan@mail.com', 'Istanbul', 'Valley Way', '34004', 'Patient'),
(11, 'Elif', 'Sahin', 'elif.sahin@mail.com', 'Ankara', 'Forest St', '06103', 'Patient'),
(12, 'Burak', 'Demir', 'burak.demir@mail.com', 'Izmir', 'Mountain Rd', '35002', 'Patient'),
(13, 'Admin', 'Manager', 'admin@hospital.com', 'Istanbul', 'Admin St', '34005', 'Admin'),
(14, 'Security', 'Officer', 'security@hospital.com', 'Istanbul', 'Security Rd', '34006', 'Security'),
(15, 'Onur', 'Bal', 'onur.bal@mail.com', 'Ankara', 'Center Rd', '06104', 'Doctor');

-- Insert multivalued phone attribute
INSERT INTO UserPhone VALUES
(1, '5551234567'), (1, '2165551000'),
(2, '5552223344'), (2, '2165552000'),
(4, '5553334455'),
(6, '5554445566'), (6, '2325553000'),
(7, '5555556789'),
(8, '5556667890');

-- Insert insurance records (>= 10 records)
INSERT INTO Insurance VALUES
(1, 'Blue Cross', 'BC2024001', 'Comprehensive'),
(2, 'Acibadem Health', 'AH2024002', 'Premium'),
(3, 'SGK (Government)', 'SGK2024001', 'Standard'),
(4, 'Allianz Turkey', 'ALZ2024001', 'Premium'),
(5, 'AXA Insurance', 'AXA2024001', 'Standard'),
(6, 'Zurich Health', 'ZUR2024001', 'Comprehensive'),
(7, 'Bupa Health', 'BUPA2024001', 'Premium'),
(8, 'DKT Insurance', 'DKT2024001', 'Standard'),
(9, 'Ray Sigorta', 'RAY2024001', 'Standard'),
(10, 'Groupama', 'GRP2024001', 'Comprehensive');

-- Insert patient records (linked to users 2,5,6,9,10,11,12)
INSERT INTO Patient VALUES
(2, 2, 'O+', '1985-03-15', 'Ayse Kaya', 1),
(5, 5, 'A+', '1990-07-22', 'Mehmet Yilmaz', 3),
(6, 6, 'B-', '1988-11-30', 'Zeynep Demir', 2),

```

```

(9, 9, 'AB+', '1995-05-14', 'Fatih Kaya', 5),
(10, 10, '0-', '1992-09-27', 'Selen Ozturk', 4),
(11, 11, 'A-', '2000-01-10', 'Emre Sahin', 6),
(12, 12, 'B+', '1987-04-18', 'Deniz Arslan', 7);

-- Insert departments
INSERT INTO Department VALUES
(1, 1, 'Cardiology', 3),
(2, 1, 'Dermatology', 2),
(3, 1, 'Neurology', 4),
(4, 2, 'Cardiology', 1),
(5, 2, 'Orthopedics', 2),
(6, 2, 'Pediatrics', 3),
(7, 3, 'Internal Medicine', 1),
(8, 3, 'Dermatology', 2);

-- Insert doctor records (users 1,4,7,8,15)
INSERT INTO Doctor VALUES
(1, 1, 1, 'Cardiology', 'LIC001234', '2024-01-10 09:00:00'),
(4, 4, 2, 'Dermatology', 'LIC001235', '2024-02-15 10:00:00'),
(7, 7, 3, 'Neurology', 'LIC001236', '2024-01-20 08:30:00'),
(8, 8, 4, 'Cardiology', 'LIC001237', '2024-03-01 14:00:00'),
(15, 15, 5, 'Orthopedics', 'LIC001238', '2024-02-10 11:00:00');

-- Insert 15+ appointments
INSERT INTO Appointment VALUES
(1, 2, 1, '2024-05-20 10:30:00', 'Completed', 'Patient shows symptoms of hypertension'),
(2, 2, 1, '2024-06-01 14:00:00', 'Scheduled', 'Follow-up appointment'),
(3, 5, 4, '2024-05-15 09:00:00', 'Completed', 'Cardiac evaluation'),
(4, 6, 4, '2024-05-22 15:30:00', 'Completed', 'Heart checkup'),
(5, 9, 7, '2024-06-05 11:00:00', 'Scheduled', 'Neurological assessment'),
(6, 10, 1, '2024-05-28 13:15:00', 'Completed', 'Blood pressure monitoring'),
(7, 11, 4, '2024-06-10 10:45:00', 'Scheduled', 'Post-MI follow-up'),
(8, 12, 15, '2024-06-12 08:00:00', 'Completed', 'Orthopedic consultation'),
(9, 2, 4, '2024-06-15 16:00:00', 'Scheduled', 'Dermatology visit'),
(10, 5, 1, '2024-06-18 09:30:00', 'Completed', 'Routine cardiac exam'),
(11, 6, 7, '2024-06-20 14:20:00', 'Scheduled', 'Neurology follow-up'),
(12, 9, 15, '2024-06-25 11:00:00', 'Completed', 'Orthopedic assessment'),
(13, 10, 4, '2024-07-01 10:00:00', 'Scheduled', 'Cardiology review'),
(14, 11, 1, '2024-07-05 15:45:00', 'Completed', 'Heart rate study'),
(15, 12, 8, '2024-07-10 09:15:00', 'Scheduled', 'Second opinion cardiology');

-- Insert prescriptions (weak entity - 15+ records)
INSERT INTO Prescription VALUES
(1, 1, 'Lisinopril', '10mg daily', 30),
(2, 1, 'Metoprolol', '25mg twice daily', 30),

```

```

(1, 2, 'Atorvastatin', '20mg daily', 60),
(1, 3, 'Aspirin', '100mg daily', 90),
(2, 3, 'Amlodipine', '5mg daily', 30),
(1, 4, 'Furosemide', '40mg daily', 30),
(1, 5, 'Digoxin', '0.25mg daily', 30),
(2, 5, 'Warfarin', '5mg daily', 30),
(1, 6, 'Clopidogrel', '75mg daily', 30),
(2, 6, 'Nitroglycerin', 'as needed', 10),
(1, 7, 'Carvedilol', '3.125mg twice daily', 30),
(1, 8, 'Losartan', '50mg daily', 30),
(2, 8, 'Hydrochlorothiazide', '12.5mg daily', 30),
(1, 9, 'Ibuprofen', '400mg thrice daily', 14),
(1, 10, 'Tramadol', '50mg as needed', 30);

-- Insert lab results
INSERT INTO LabResult VALUES
(1, 1, 'Blood Pressure', '140/90 mmHg', '2024-05-20'),
(2, 1, 'Cholesterol', '240 mg/dL', '2024-05-20'),
(3, 3, 'ECG', 'Normal sinus rhythm', '2024-05-15'),
(4, 4, 'Troponin', '0.02 ng/mL', '2024-05-22'),
(5, 5, 'MRI Brain', 'No abnormalities', '2024-06-05'),
(6, 6, 'Hemoglobin', '13.5 g/dL', '2024-05-28'),
(7, 7, 'BNP', '45 pg/mL', '2024-06-10'),
(8, 8, 'X-Ray', 'Normal', '2024-06-12'),
(9, 10, 'Blood Glucose', '120 mg/dL', '2024-06-18'),
(10, 12, 'CT Scan', 'Findings noted', '2024-06-25'),
(11, 14, 'Electrocardiogram', 'Abnormal - T wave', '2024-07-05');

-- Insert billing records
INSERT INTO Billing VALUES
(1, 2, 800.00, 'Paid', '2024-05-20'),
(2, 5, 750.00, 'Pending', '2024-05-15'),
(3, 6, 900.00, 'Paid', '2024-05-22'),
(4, 9, 650.00, 'Unpaid', '2024-06-05'),
(5, 10, 800.00, 'Paid', '2024-05-28'),
(6, 11, 750.00, 'Pending', '2024-06-10'),
(7, 12, 700.00, 'Paid', '2024-06-12'),
(8, 2, 800.00, 'Unpaid', '2024-06-15'),
(9, 5, 900.00, 'Paid', '2024-06-18'),
(10, 6, 750.00, 'Pending', '2024-06-20');

-- Insert medical devices
INSERT INTO MedicalDevice VALUES
(1, 1, 'Cardiac Monitor', '4.2.1', 'High'),
(2, 1, 'ECG Machine', '3.1.0', 'Medium'),
(3, 1, 'Defibrillator', '5.0.0', 'Critical'),

```



```

(4, 2, 'MRI Scanner', '2.5.1', 'Critical'),
(5, 2, 'CT Scanner', '3.8.0', 'High'),
(6, 3, 'Ultrasound', '2.1.0', 'Medium');

-- Insert security events
INSERT INTO SecurityEvent VALUES
(1, 1, 1, 'Device Access', 'INFO', '192.168.1.100', '2024-05-20 10:00:00'),
(2, 2, 2, 'Data Query', 'INFO', '192.168.1.101', '2024-05-20 11:30:00'),
(3, 3, 1, 'Configuration Change', 'WARNING', '192.168.1.102', '2024-05-20 14:45:00'),
(4, 1, 3, 'Firmware Update Pending', 'CRITICAL', '192.168.1.100', '2024-05-21 09:00:00'),
(5, 4, 4, 'Unauthorized Access Attempt', 'CRITICAL', '203.0.113.50', '2024-05-21 15:30:00');

```

6.3 ALTER TABLE Operations

```

-- Add a new column for patient registration date
ALTER TABLE Patient ADD COLUMN RegistrationDate DATE DEFAULT CURRENT_DATE;

-- Modify the Insurance table to add coverage percentage
ALTER TABLE Insurance ADD COLUMN CoveragePct INT;

-- Add check constraint on billing amounts
ALTER TABLE Billing ADD CONSTRAINT chk_amount CHECK (Amount > 0);

-- Add check constraint on doctor consultation fee
ALTER TABLE Doctor ADD CONSTRAINT chk_fee CHECK (
    (SELECT Specialty FROM Doctor) IS NOT NULL AND
    (SELECT LicenseNumber FROM Doctor) IS NOT NULL
);

-- Create a check for valid appointment status
ALTER TABLE Appointment ADD CONSTRAINT chk_status CHECK (Status IN ('Scheduled', 'Completed'));

-- Modify column type for better precision
ALTER TABLE MedicalDevice MODIFY COLUMN RiskLevel ENUM('Low', 'Medium', 'High', 'Critical');

-- Add index on frequently queried columns
ALTER TABLE Appointment ADD INDEX idx_appointment_date (AppointmentDate);
ALTER TABLE Appointment ADD INDEX idx_patient_doctor (PatientID, DoctorID);

-- Add a trigger-ready audit column (optional, some systems)
ALTER TABLE SecurityEvent ADD COLUMN AuditLog VARCHAR(500);

```

8 7. Advanced SQL Queries

Query 1: Doctors with more than 5 appointments

```
SELECT d.DoctorID, u.FirstName, u.LastName, COUNT(a.AppointmentID) as appointment_count
FROM Doctor d
JOIN User u ON d.UserID = u.UserID
JOIN Appointment a ON d.DoctorID = a.DoctorID
GROUP BY d.DoctorID, u.FirstName, u.LastName
HAVING COUNT(a.AppointmentID) > 5
ORDER BY appointment_count DESC;
```

Query 2: Patients with no appointments scheduled

```
SELECT p.PatientID, u.FirstName, u.LastName
FROM Patient p
JOIN User u ON p.UserID = u.UserID
LEFT JOIN Appointment a ON p.PatientID = a.PatientID
WHERE a.AppointmentID IS NULL;
```

Query 3: Most common diagnoses by department

```
SELECT d.DepartmentName, a.DiagnosisSummary, COUNT(*) as frequency
FROM Appointment a
JOIN Doctor dr ON a.DoctorID = dr.DoctorID
JOIN Department d ON dr.DepartmentID = d.DepartmentID
WHERE a.DiagnosisSummary IS NOT NULL
GROUP BY d.DepartmentName, a.DiagnosisSummary
ORDER BY d.DepartmentName, frequency DESC;
```

Query 4: Patients with incomplete billing

```
SELECT p.PatientID, u.FirstName, u.LastName, b.Amount, b.PaymentStatus
FROM Patient p
JOIN User u ON p.UserID = u.UserID
JOIN Billing b ON p.PatientID = b.PatientID
WHERE b.PaymentStatus != 'Paid'
ORDER BY b.BillingDate DESC;
```

Query 5: Appointments with prescriptions and lab results

```
SELECT a.AppointmentID, p.PatientID, u.FirstName,
       COUNT(DISTINCT pr.PrescriptionNo) as prescription_count,
       COUNT(DISTINCT lr.ResultID) as lab_result_count
FROM Appointment a
JOIN Patient p ON a.PatientID = p.PatientID
JOIN User u ON p.UserID = u.UserID
```

```

LEFT JOIN Prescription pr ON a.AppointmentID = pr.AppointmentID
LEFT JOIN LabResult lr ON a.AppointmentID = lr.AppointmentID
GROUP BY a.AppointmentID, p.PatientID, u.FirstName
HAVING COUNT(DISTINCT pr.PrescriptionNo) > 0
      AND COUNT(DISTINCT lr.ResultID) > 0;

```

Query 6: Average appointments per doctor by department

```

SELECT d.DepartmentName, AVG(doctor_appointments) as avg_appointments
FROM Department d
JOIN (
    SELECT dr.DepartmentID, dr.DoctorID, COUNT(a.AppointmentID) as doctor_appointments
    FROM Doctor dr
    LEFT JOIN Appointment a ON dr.DoctorID = a.DoctorID
    GROUP BY dr.DepartmentID, dr.DoctorID
) subquery ON d.DepartmentID = subquery.DepartmentID
GROUP BY d.DepartmentName;

```

Query 7: Security events by severity

```

SELECT u.FirstName, u.LastName, se.Severity, COUNT(se.EventID) as event_count
FROM SecurityEvent se
JOIN User u ON se.UserID = u.UserID
WHERE se.EventTimestamp >= DATE_SUB(NOW(), INTERVAL 30 DAY)
GROUP BY u.UserID, u.FirstName, u.LastName, se.Severity
HAVING COUNT(se.EventID) > 2
ORDER BY se.Severity DESC, event_count DESC;

```

Query 8: Patients' insurance coverage details

```

SELECT p.PatientID, u.FirstName, u.LastName,
       i.ProviderName, i.CoverageType,
       COUNT(a.AppointmentID) as appointment_count
FROM Patient p
JOIN User u ON p.UserID = u.UserID
JOIN Insurance i ON p.InsuranceID = i.InsuranceID
LEFT JOIN Appointment a ON p.PatientID = a.PatientID
GROUP BY p.PatientID, u.FirstName, u.LastName,
         i.ProviderName, i.CoverageType;

```

Query 9: Medical devices with high risk needing firmware updates

```

SELECT h.HospitalName, md.DeviceType, md.FirmwareVersion, md.RiskLevel
FROM MedicalDevice md
JOIN Hospital h ON md.HospitalID = h.HospitalID

```

```
WHERE md.RiskLevel IN ('High', 'Critical')
ORDER BY h.HospitalName, md.RiskLevel DESC;
```

Query 10: Prescription refill analysis

```
SELECT
    pr.MedicineName,
    COUNT(pr.PrescriptionNo) as prescription_count,
    AVG(pr.DurationDays) as avg_duration,
    MAX(a.AppointmentDate) as last_prescribed
FROM Prescription pr
JOIN Appointment a ON pr.AppointmentID = a.AppointmentID
GROUP BY pr.MedicineName
ORDER BY prescription_count DESC;
```

9 8. Relational Algebra and Calculus

8.1 Relational Algebra Queries

Query 1: Find all patients from Istanbul

$$\pi_{PatientID, FirstName, LastName}(\sigma_{City='Istanbul'}(User \bowtie Patient))$$

Query 2: Appointments scheduled for May 2024

$$\pi_{AppointmentID, PatientID, DoctorID, Status}(\sigma_{AppointmentDate \geq '2024-05-01' \wedge AppointmentDate < '2024-06-01'}(Appointment))$$

Query 3: Doctors in Cardiology department

$$\pi_{DoctorID, FirstName, LastName}(Doctor \bowtie_{DepartmentID} (\sigma_{DepartmentName='Cardiology'}(Department))) \bowtie_{UserID} User$$

8.2 Tuple Relational Calculus Queries

Query 1: Find all patients with blood type O+

$$\{t \mid t \in Patient \wedge t.BloodType = 'O+\}'$$

Query 2: Find doctors and their department names

$$\{(d, d_name, dept_name) \mid Doctor(d) \wedge User(u) \wedge d.UserID = u.id \wedge Department(dept) \wedge d.DepartmentID = dept.ID\}$$

Query 3: Find appointments with prescriptions

$$\{a.AppointmentID \mid Appointment(a) \wedge \exists p (Prescription(p) \wedge a.AppointmentID = p.AppointmentID)\}$$

10 9. Indexing and File Structures

9.1 Index Strategy

The following indexes are created for performance optimization:

- **Appointment(PatientID)**: Frequently used in WHERE and JOIN clauses for patient lookups
- **Appointment(DoctorID)**: Supports queries filtering appointments by doctor
- **Appointment(AppointmentDate)**: Enables range queries on appointment dates
- **SecurityEvent(UserID)**: Optimizes SIEM queries monitoring user activity
- **SecurityEvent(EventTimestamp)**: Supports time-based filtering and auditing

9.2 Index Comparison: Indexed vs Non-Indexed

Scenario: Query patients with appointments in May 2024

Without Index on Appointment(PatientID, AppointmentDate):

- Full table scan of Appointment table: $O(n)$ where n = total appointments
- Estimated rows examined: 50,000 (for large datasets)
- Query time: 5-10 seconds

With B+ Tree Index on Appointment(PatientID, AppointmentDate):

- Index lookup: $O(\log n)$
- Rows examined: 50-100 (range scan within index)
- Query time: 50-200 milliseconds
- **Performance improvement: 25-100x faster**

9.3 B+ Tree Index Structure for Appointment

Leaf Nodes: [PatientID, AppointmentDate, Pointer to row]

Example:

```
[1, 2024-05-10, Row1000]
[1, 2024-05-20, Row1001]
[2, 2024-05-15, Row1002]
...
```

[100, 2024-06-01, Row5000]

Interior Nodes route to appropriate leaf ranges.

9.4 Hash Index for Exact Lookups

Index: User(Email)

- Hash function distributes emails into buckets
- Average lookup time: $O(1)$
- Use case: User login by email

9.5 Index Maintenance Considerations

- **Write overhead:** Indexes slow INSERT/UPDATE/DELETE by 10-20%
- **Storage cost:** Each index requires ~20% additional disk space
- **Maintenance:** Indexes on frequently updated columns (e.g., Status) should be monitored
- **Strategy:** Index hot columns (PatientID, DoctorID, AppointmentDate) only

9.6 Performance Summary

Query Type	Without Index	With Index
Single patient lookup	5-10 sec	10-50 ms
Date range query (month)	8-15 sec	50-200 ms
Doctor’s appointments	10-20 sec	100-300 ms
Security event filtering	20+ sec	50-100 ms
Indexing provides 50-200x improvement for common queries in large healthcare datasets.		